

The Semantic Web ECS 735P

Mini Project

Student Name: Semil Halani

Student Number: 210147012

1. Introduction

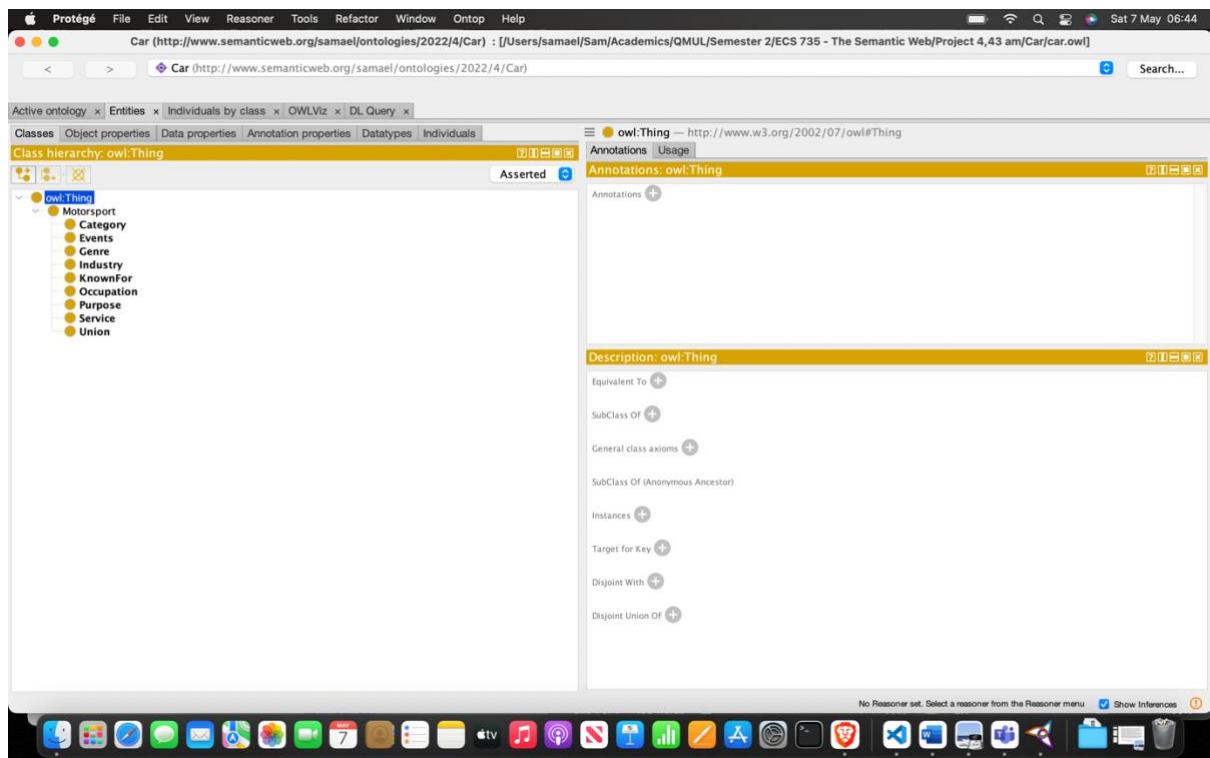
The main purpose of this mini project for the module *The Semantic Web* is to define an ontology which is to be used to fill the data from an external source and apply a few queries for the ontology to illustrate the correct workflow and the combined information offered by it.

Basic Task

Initially, I looked for the best kind of fit for the domain and the source of data which would be used for building the ontology for this project. After a lot of efforts of doing research online, I decided to select the data related to *Motorsport Cars*, from the online data resource, *DBpedia*. I referred a lot of different entries of the domain, which possesses few useful information in order to construct the ontology for my project. Furthermore, I used the software application named Protégé, version 5.5.0, for defining the ontology structure for my project, which resulted in the file named *car_basic.owl*.

Class Hierarchy

According to the data from the dataset on the website, *DBpedia*, I discovered the structure of the information given and then arranged them in a proper way. Moreover, I created 10 classes for the same, as shown below:



As shown in the image, there are a few classes such as *Category*, *Events*, *Industry*, *Service* etc. *MOTORSPORT* is the top most class and then the other classes are beneath it. All the other classes are the subclasses of the class *MOTORSPORT*. The *Service* class just includes 2 records and the *Union* class has just 1 record, whereas the *Genre*, *Industry* and other classes have a lot of other varied data for those respective classes.

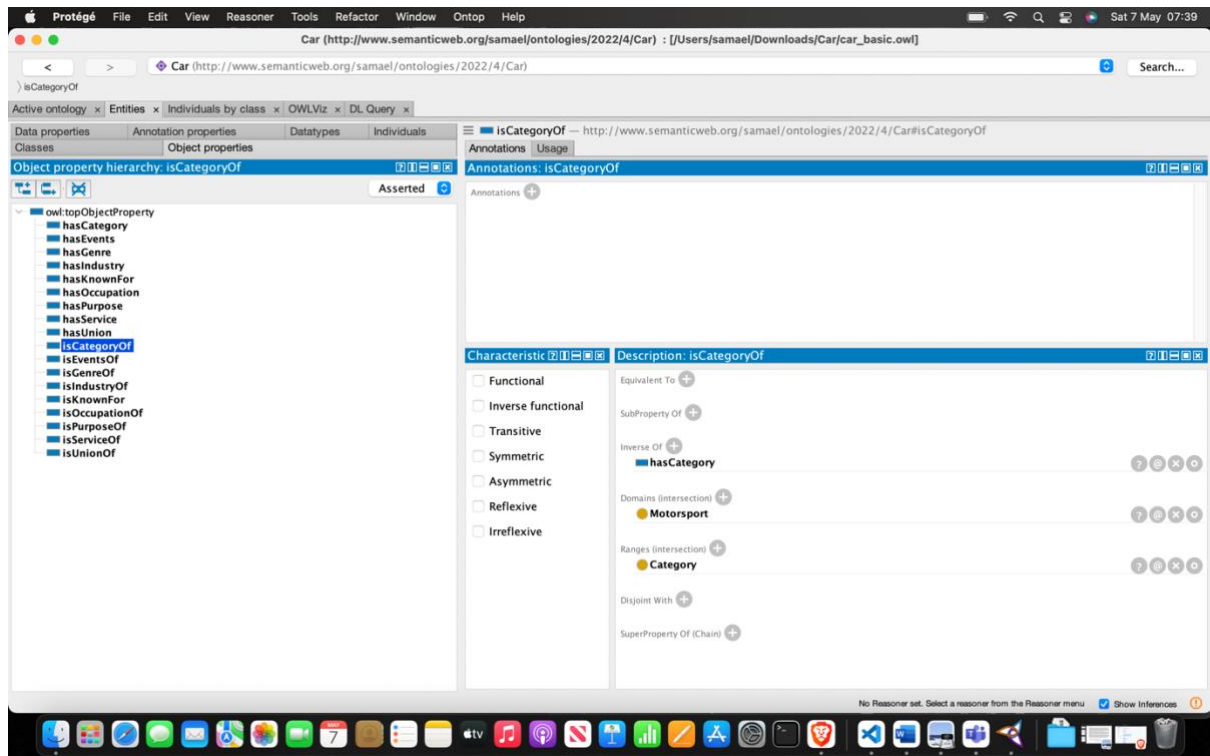
Class	Asserted information
MOTORSPORT	N/A
Category	N/A
Events	N/A
Genre	N/A
Industry	N/A
KnownFor	N/A
Occupation	N/A
Purpose	N/A
Service	N/A
Union	N/A

Table 1. Asserted information for each class in the *MOTORSPORT* Cars ontology.

2. Concepts and Properties

Object Properties:

To illustrate the relationship between the classes, I created a few object properties which would relate the classes as shown below:



With respect to the object properties discussed above, I went through the process of determining whether the object properties are functional, inversely functional, symmetric etc. For instance, consider the object property *isGenreOf*; it is Asymmetric because every Motorsport car is a genre of the respective genre whereas vice versa doesn't hold true.

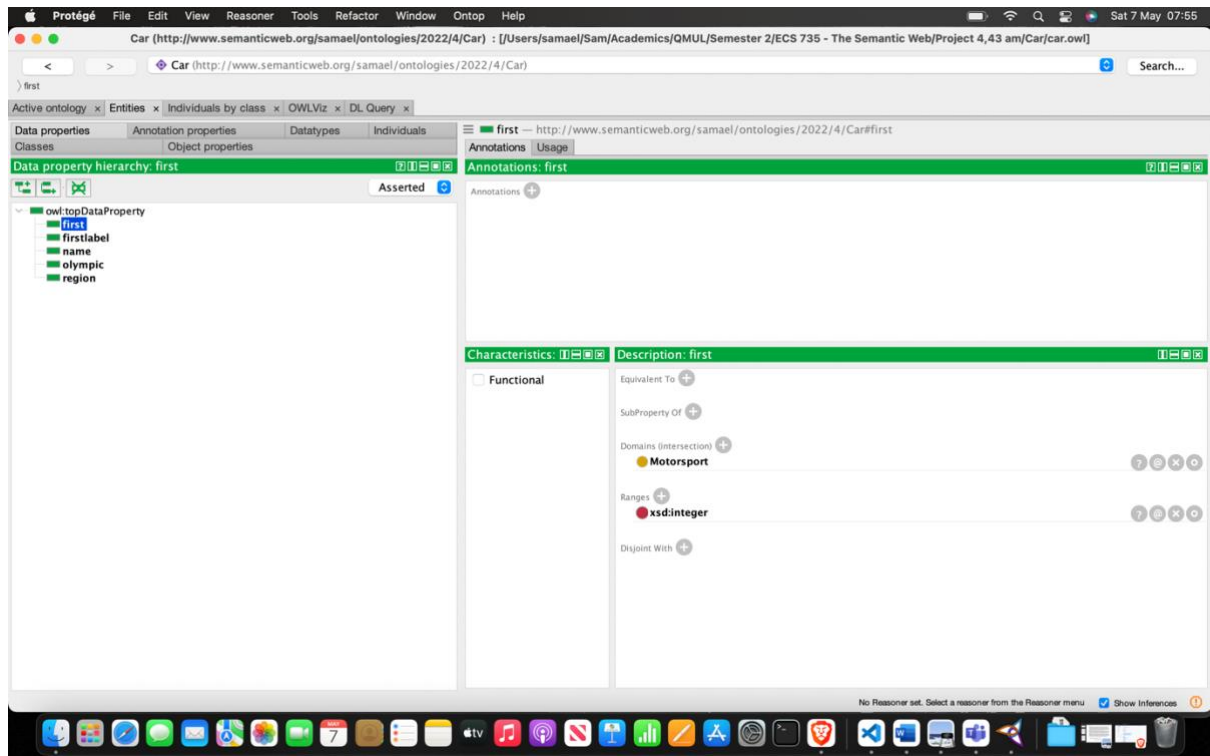
Also, *isGenreOf* is an inverse property for the object property *hasGenre*. It works in similar way for most of the object properties.

We asserted the following object properties:

- hasUnion: Motorsport cars has different unions
- isCategoryOf: Motorsport cars is a different category
- isEventsOf: Motorsport cars is related to different events
- isGenreOf: Motorsport cars is a genre of different instances
- isIndustryOf: Motorsport cars is industry of different instances
- isKnownFor: Motorsport cars is known for various stuff
- isOccupationOf: Motorsport cars is an Occupation of a lot of people
- isPurposeOf: Motorsport cars is a purpose for a lot of instances
- isServiceOf: Motorsport cars is a service for a few different instances

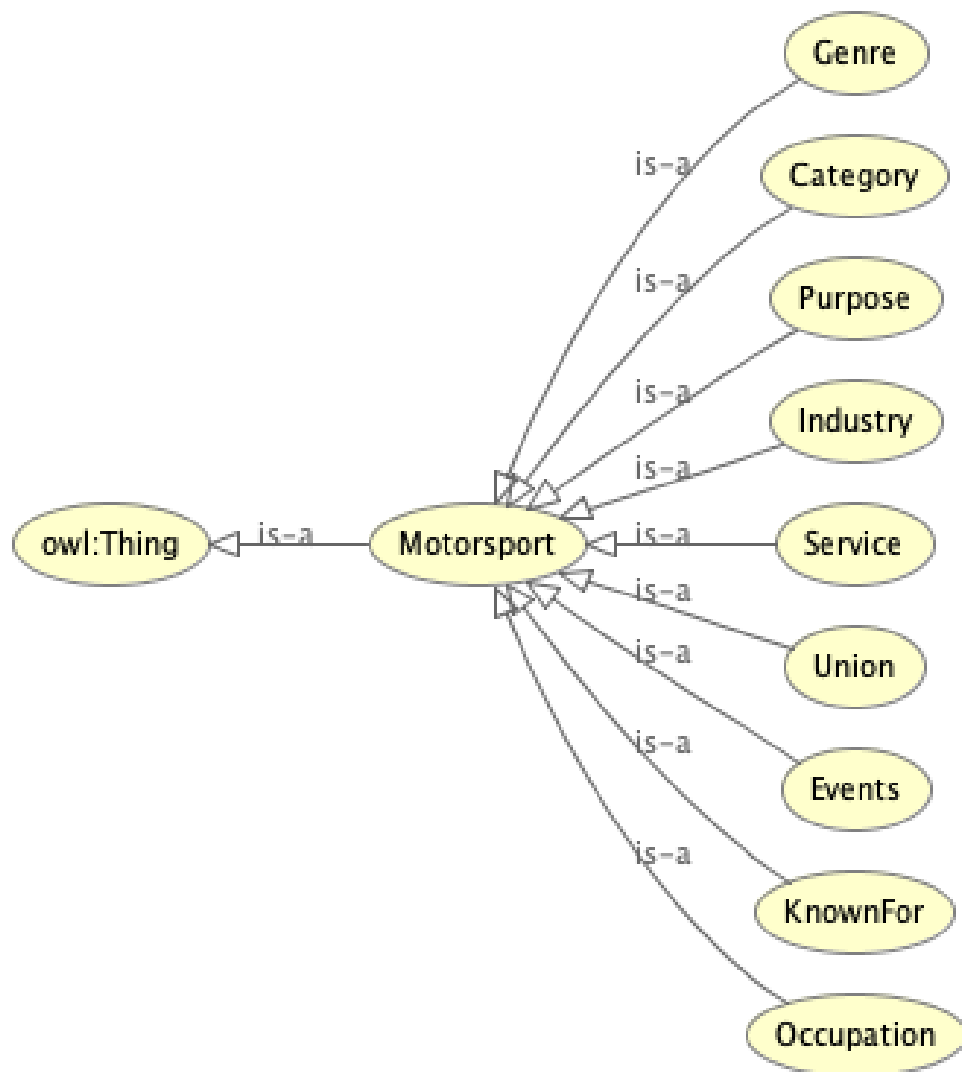
Data Properties:

For the mentioned dataset, I created 5 data properties as shown below:



For the above 5 data properties,

- first: Motorsport car has a year when it was first released. The first is an integer.
- name: Motorsport car has a name. The name is a string.
- region: Motorsport car has a region. The region is a string.
- olympic: Motorsport car has a year when it appeared in the Olympic for the first time. The olympic is an integer.
- firstlabel: Motorsport car has a firstlabel. The firstlabel is a string.



3. Asserted Individuals

Class	Individuals
Service	MotorSport_Vision, Racelogic
Genre	Supercars Life, Sniff Petrol, ...
Category	Auto_Action, Avto_Plus, Auto Motor und Sport, ...
Industry	Alpinestars, AMR_GP, AGV_Sports_Group, Abt Sportsline, ...
KnownFor	Antonio_Maeso, Charlie_Christina_Martin, Lee McKenzie, ...
Events	Alexandra_Park_Street_Circuit
Occupation	Alfieri_Maserati, Ralph_Bellamy_(racing_car_designer), Bill Boddy, ...
Purpose	FIA_World_Motor_Sport_Council, ASN_Canada_FIA, Fédération Internationale de l'Automobile, ...
Union	Alexandra_Park_Street_Circuit

For a full list of Individuals, kindly refer to the Ontology file.

Query Data

The query is shown below in the query_basic.py file, which queries all the motorsports cars according to their genre:

```

1  import rdflib
2
3  # This simple query grabs all motorsport cars with it's genre and service and sorts it out according to the genre.
4  query = """
5  PREFIX ma: <http://www.semanticweb.org/samael/ontologies/2022/4/Car#>
6  PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
7  SELECT DISTINCT ?genre ?service
8  WHERE
9  {
10     ?motors rdf:type ma:Motorsport .
11     ?motors ma:isGenreOf ?genre .
12     ?genre rdf:type ma:Genre .
13     ?motors ma:isServiceOf ?service .
14     ?service rdf:type ma:Service .
15  } GROUP BY ?genre """
16
17
18 # Create an empty RDF graph and then parse our generated ontology into it.
19 g = rdflib.Graph()
20 g.parse("car_basic.owl", "xml")
21
22 print("graph has %s statements.\n" % len(g))
23
24 # formatting strings used in Python to nicely format a table and print the respective data.
25 print("Genre", "\t", "Service")
26 for x, y in g.query(query):
27     print(x, "\t", y)
28
29 print("")
30 print(" Try editing the code to run this query against 'car.owl' instead.")
31 print(" Notice how the query runs and a few results are returned. When")
32 print(" your ontology it can be handy to define a few individuals so you can")
33 print(" test your queries and perform some sanity checks. That way you can focus")
34 print(" your attention on writing the proper queries to populate the rest of")
35 print(" your ontology now you know what your individuals should look like.")
36 print("")
37

```

Result:

```

samael@Semils-Air Car % python3 query_basic.py
graph has 307 statements.

Genre
http://dbpedia.org/resource/Auto_Action
http://dbpedia.org/resource/Racer_(magazine)
http://dbpedia.org/resource/Parkside_Media
http://dbpedia.org/resource/Inside_Supercars
http://dbpedia.org/resource/Grassroots_Motorsports
http://dbpedia.org/resource/Avto_Plus
http://dbpedia.org/resource/Sniff_Petrol
http://dbpedia.org/resource/In_Pit_Lane
http://dbpedia.org/resource/Motoring_Today
http://dbpedia.org/resource/Motor_Sport_(magazine)
http://dbpedia.org/resource/The_Pits_(TV_series)
http://dbpedia.org/resource/Auto_Motor_und_Sport
http://dbpedia.org/resource/Autosport_International
http://dbpedia.org/resource/Supercars_Life
http://dbpedia.org/resource/Mobil_1_The_Grid
http://dbpedia.org/resource/George_Levy
http://dbpedia.org/resource/Torque_(magazine)
http://dbpedia.org/resource/Bangla_Bangers
http://dbpedia.org/resource/Shannons_Legends_of_Motorsport
http://dbpedia.org/resource/RPM_(TV_series)

Service
http://dbpedia.org/resource/MotorSport_Vision
http://dbpedia.org/resource/MotorSport_Vision
http://dbpedia.org/resource/MotorSport_Vision
http://dbpedia.org/resource/MotorSport_Vision
http://dbpedia.org/resource/MotorSport_Vision
http://dbpedia.org/resource/MotorSport_Vision
http://dbpedia.org/resource/MotorSport_Vision
http://dbpedia.org/resource/MotorSport_Vision
http://dbpedia.org/resource/MotorSport_Vision
http://dbpedia.org/resource/MotorSport_Vision
http://dbpedia.org/resource/MotorSport_Vision
http://dbpedia.org/resource/MotorSport_Vision
http://dbpedia.org/resource/MotorSport_Vision
http://dbpedia.org/resource/MotorSport_Vision
http://dbpedia.org/resource/MotorSport_Vision
http://dbpedia.org/resource/MotorSport_Vision
http://dbpedia.org/resource/MotorSport_Vision
http://dbpedia.org/resource/MotorSport_Vision
http://dbpedia.org/resource/MotorSport_Vision
http://dbpedia.org/resource/MotorSport_Vision

Try editing the code to run this query against 'car.owl' instead.
Notice how the query runs and a few results are returned. When
your ontology it can be handy to define a few individuals so you can
test your queries and perform some sanity checks. That way you can focus
your attention on writing the proper queries to populate the rest of
your ontology now you know what your individuals should look like.

samael@Semils-Air Car %

```

4. Issues

There was a lot of issues that were faced while looking for the information and creating the ontology. For example, as there was observed a lot of inconsistency of the data obtained through the data resource, some had enough data while some did not, I had to manually select which classes should be created. Furthermore, a few individuals have an incorrect data type which is why I had to format the ontology in a more structured and correct manner. Furthermore, the reasoner takes a lot of time which results in a slow and delayed process.

5. Conclusion

We can get a lot more information from the initial statements asserted into the ontology if we use inference. Furthermore, it can be noted that the efficiency of the reasoning process needs to be worked upon too, as the reasoners are often too time consuming, which restricts us from a lot of other rule definitions and thus limits our information gain overall.

Ontology metrics:	
Metrics	
Axiom	306
Logical axiom count	273
Declaration axioms count	33
Class count	10
Object property count	18
Data property count	5
Individual count	89
Annotation Property count	0
Class axioms	
SubClassOf	9
EquivalentClasses	0
DisjointClasses	0
GCI count	0
Hidden GCI Count	0
Object property axioms	
SubObjectPropertyOf	0
EquivalentObjectProperties	0
InverseObjectProperties	9
DisjointObjectProperties	0
FunctionalObjectProperty	0
InverseFunctionalObjectProperty	18
TransitiveObjectProperty	0
SymmetricObjectProperty	0
AsymmetricObjectProperty	18
ReflexiveObjectProperty	0
IrreflexiveObjectProperty	0
ObjectPropertyDomain	9
ObjectPropertyRange	9
SubPropertyChainOf	0
Data property axioms	
SubDataPropertyOf	0
EquivalentDataProperties	0
DisjointDataProperties	0

Data property axioms	
SubDataPropertyOf	0
EquivalentDataProperties	0
DisjointDataProperties	0
FunctionalDataProperty	0
DataPropertyDomain	5
DataPropertyRange	5
Individual axioms	
ClassAssertion	96
ObjectPropertyAssertion	95
DataPropertyAssertion	0
NegativeObjectPropertyAssertion	0
NegativeDataPropertyAssertion	0
SameIndividual	0
DifferentIndividuals	0
Annotation axioms	
AnnotationAssertion	0
AnnotationPropertyDomain	0
AnnotationPropertyRangeOf	0

6. References

- <https://dbpedia.org/page/Motorsport>
- [https://dbpedia.org/page/Auto Motor und Sport](https://dbpedia.org/page/Auto_Motor_und_Sport)
- <https://dbpedia.org/page/Car>